DATA305 / DATA475 · Machine Learning Methods

Training Deep Networks II

Optimizers · AdamW · LR Scheduling · Dropout · L1/L2 · Keras Tuner

Lecture 5

Week 7 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda

Five sections · CIFAR-10 running example

01 Faster Optimizers

SGD → Momentum → RMSProp → Adam → AdamW

02 LR Scheduling

Step · Cosine · 1-cycle · ReduceLROnPlateau · Warmup

03 Regularization

Overfitting diagnosis · L1/L2 · Dropout · Max-Norm · Early Stopping

04 Hyperparameter Tuning

Keras Tuner · RandomSearch · Hyperband · Model selection

05 CIFAR-10 Experiments

Ablation study · Practical defaults · Decision guide

Dataset: keras.datasets.cifar10 (50 000 train / 10 000 test, 32×32 RGB) · Geron Ch. 11 pp. 357–430

Lecture 4 → Lecture 5

In L4 we made gradients stable. In L5 we assume gradients can now flow, and ask a new question: how do we turn those gradients into fast, stable, and well-generalizing learning?

Lecture 4 — Covered

- Vanishing & exploding gradients — chain rule, sigmoid saturation
- He & Glorot initialization — matched to activation function
- Modern activations: ReLU, LeakyReLU, ELU, GELU
- Batch Normalization: Dense → BN → Activation
- Gradient Clipping: clipnorm = 1.0 as safe default

Lecture 5 — Today

- Faster optimizers: SGD → Momentum → RMSProp → Adam → AdamW
- LR Scheduling: warmup, cosine, 1-cycle, ReduceLROnPlateau
- AdamW: why decoupled weight decay matters
- CIFAR-10 training dynamics: compare optimizers and schedules
- Practical defaults: optimizer + schedule + regularization

01

Faster Optimizers

SGD and its limits · Momentum & NAG · RMSProp · Adam · AdamW

Mini-batch SGD — The Starting Point

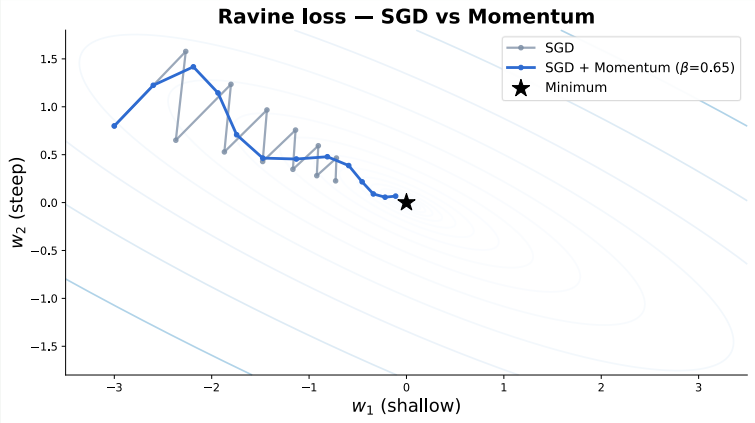
Three variants of gradient descent:

Batch GD:	full dataset each step — stable but very slow
SGD:	one sample per step — noisy, fast, but high variance
Mini-batch SGD:	standard practice — batch of 32–256 samples

Limitations of vanilla mini-batch SGD:

- Oscillates across narrow "ravine" dimensions
- Single global learning rate — no per-parameter adaptation
- Sensitive to learning rate choice; often converges more slowly initially than adaptive methods

SGD can reach very good solutions but often requires more careful tuning and may converge more slowly at the start than adaptive methods.



Momentum & Nesterov Accelerated Gradient

Momentum

$$m \leftarrow \beta\, m - \eta\, \nabla L$$
$$w \leftarrow w + m$$

- Accumulates a velocity vector in gradient direction
- Dampens oscillations in ravine-shaped loss landscapes
- $\beta = 0.9$ is the standard default
- Keras: SGD(momentum=0.9)

Intuition: ball rolling downhill — accumulates velocity in consistent gradient directions. Typical $\beta = 0.9$.

Nesterov Accelerated Gradient (NAG)

$$m \leftarrow \beta\, m - \eta\, \nabla L(w + \beta\, m)$$
$$w \leftarrow w + m$$

- Computes gradient at projected position $w + \beta\, m$
- Often slightly faster convergence than plain momentum
- Keras: SGD(momentum=0.9, nesterov=True)
- Typical default: $\beta = 0.9$

Peeks ahead before computing the gradient — often gives a slightly better correction than plain momentum, especially in curved regions.

RMSProp & Adam — Adaptive Learning Rates

⚠ AdaGrad (precursor): accumulates all past squared gradients → denominator grows without bound → learning rate collapses to zero. Fix: use exponential moving average.

RMSProp

$$s \leftarrow \rho s + (1-\rho)(\nabla L)^2$$
$$w \leftarrow w - \eta \nabla L / \sqrt{(s + \varepsilon)}$$

- Exponential moving average of squared gradients
- Denominator stays bounded → lr remains active
- ρ = 0.9 (decay rate), ε ≈ 1e-7
- Often works well on noisy problems; historically popular for RNNs
- Keras: RMSprop(learning_rate=1e-3, rho=0.9)

Adam — Adaptive Moment Estimation

$$m \leftarrow \beta_1 m + (1-\beta_1)\nabla L$$
$$v \leftarrow \beta_2 v + (1-\beta_2)(\nabla L)^2$$
$$\hat{m} = m/(1-\beta_1{}^t), \quad \hat{v} = v/(1-\beta_2{}^t) \quad [bias-corrected]$$
$$w \leftarrow w - \eta\, \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$$

- Combines Momentum (1st moment) + RMSProp (2nd moment)
- Bias correction compensates for m and v starting at 0 (early estimates less biased)
- β_1=0.9, β_2=0.999, ε=1e-7 — rarely need to change
- Keras: Adam(learning_rate=1e-3, clipnorm=1.0)

AdamW — Decoupled Weight Decay

Problem: With SGD, L2 regularization and weight decay are equivalent. With Adam, they are NOT.

Adam + L2 Regularization

$$\nabla L = \nabla L + 2\lambda w \quad (augmented\ gradient)$$

Adam update uses ∇L — decay couples with adaptive lr

- L2 penalty is folded into the gradient
- Adam's adaptive scaling then modifies the effective weight decay
- The actual decay magnitude depends on gradient history
- NOT equivalent to proper weight decay with Adam

AdamW — Decoupled Weight Decay ✓

$$w \leftarrow w - \eta\, \hat{m} / (\sqrt{\hat{v}} + \varepsilon) - \eta\, \lambda\, w$$

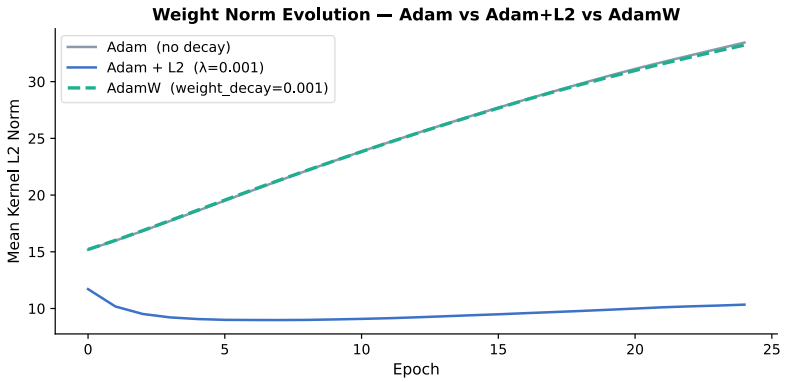
$$[Adam\ step] \qquad [weight\ decay, decoupled]$$

- Weight decay applied directly to weights, not gradients
- Decoupled from adaptive update → consistent, predictable decay
- Modern preferred default in many training setups
- Keras: AdamW(learning_rate=1e-3, weight_decay=1e-4)

AdamW vs. Adam + L2

Empirical evidence: Adam + L2 and AdamW do not behave the same in practice.

Weight Norm Evolution



Metric: mean kernel L2 norm across epochs.

Key Interpretation

- Adam + L2 sharply suppresses the mean kernel norm, while AdamW stays much closer to plain Adam in this run.
- That does not mean AdamW = Adam. It means AdamW is not injecting the same gradient-coupled shrinkage as Adam + L2.
- With Adam, adding an L2 term changes the adaptive gradient update. AdamW instead applies decay directly to the weights.
- So “Adam + L2” and “AdamW” can produce very different training dynamics, even when λ / wd are set to the same nominal value.

Optimizer Comparison — Practical Guide

Optimizer	Advantages	Disadvantages	Use when…
SGD + Momentum	Excellent generalization; precise control	Requires careful LR tuning; converges more slowly initially	Baseline / vision models from scratch
RMSProp	Handles noisy gradients well	Less commonly used than Adam today	RNNs; older architectures
Adam	Fast convergence; robust defaults	Can generalize slightly worse than SGD+Momentum	Strong default for most tasks
AdamW ★	Adam speed + proper weight decay	One extra hyperparameter (λ)	**Modern preferred default**

Key intuitions: Momentum stores direction · RMSProp rescales by recent gradient magnitudes · Adam = both · AdamW = Adam + clean weight decay

02

Learning Rate Scheduling

Step · Exponential · Cosine · 1-cycle policy · ReduceLROnPlateau · Warmup

Why Learning Rate Scheduling?

LR Too Large

Large oscillations, may diverge.

NaN loss common.

LR Fixed (Small)

Stable but very slow.

Rarely optimal for full training run.

LR Scheduled

Often faster progress early.

Finer convergence later.

A fixed learning rate is rarely optimal for the full training run. Often it helps to start relatively high, then reduce for fine-grained convergence.

Scheduling Strategies

Step / Piecewise

```
lr × 0.1  every N epochs
```

- Simple, predictable schedule
- Common in ResNet training
- Requires manual epoch selection

Exponential Decay

```
lr₀ × rate^(step/steps)
```

- Smooth, continuous decrease
- LearningRateScheduler callback
- Good general-purpose baseline

Cosine Annealing

```
lr_min + ½(lr_max−lr_min)(1+cos(πt/T))
```

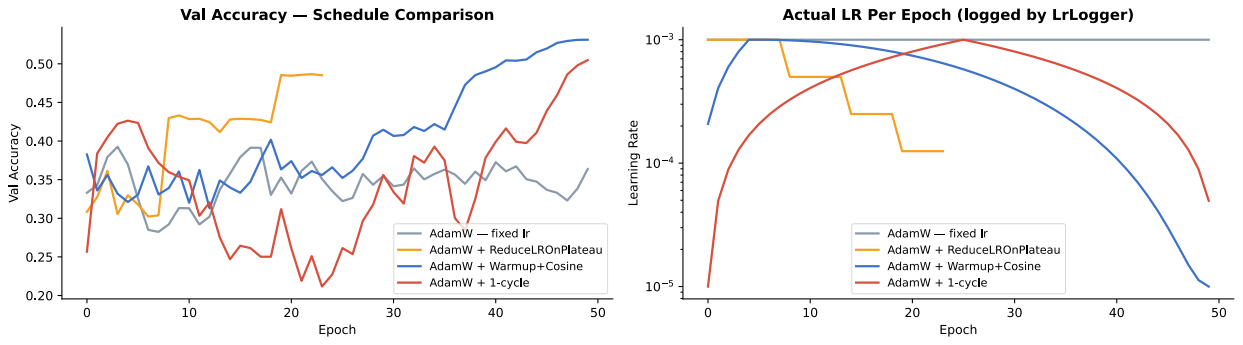
- Smooth decay following cosine curve
- Often combined with warmup
- Common in many modern training setups

ReduceLROnPlateau

```
if no improvement: lr *= factor
```

- Reactive — monitors val_loss
- factor=0.5, patience=5
- Already seen in Lecture 3

LR Scheduling — Empirical Results (CIFAR-10 MLP)



Fixed LR **~35–40%**

Rarely optimal; leaves performance on the table

ReduceLROnPlateau **~48–52%**

Reactive: cuts LR when val_loss plateaus. Reliable.

Warmup + Cosine **~52–54%**

Linear warmup → cosine decay. ✓ Recommended default

1-cycle **~50–52%**

LR ↑ then ↓, momentum inverse. Can reach good accuracy faster.

Key result: any schedule beats a fixed LR. Warmup+Cosine is strongest here; ReduceLROnPlateau is a simple reactive baseline.

Warmup + Cosine Decay — Keras


```python
import math
from tensorflow.keras.callbacks import LearningRateScheduler

WARMUP_EPOCHS = 5
LR_START = 1e-4
LR_MAX = 1e-3
TOTAL_EPOCHS = 50

def lr_schedule(epoch, lr):
    if epoch < WARMUP_EPOCHS:    # linear warmup
        progress = (epoch + 1) / WARMUP_EPOCHS
        return LR_START + progress * (LR_MAX - LR_START)
    progress = (epoch - WARMUP_EPOCHS) / max(1, TOTAL_EPOCHS - WARMUP_EPOCHS - 1)
    return LR_START + 0.5 * (LR_MAX - LR_START) * (1 + math.cos(math.pi * progress))

model.fit(..., callbacks=[LearningRateScheduler(lr_schedule)])
```

Linear Warmup: Start low (1e-4), rise to lr_max over 5 epochs. Prevents instability with fresh BN layers or new heads.

Cosine Decay: Smoothly anneals from lr_max back to lr_min. Often smoother than step decay and widely used in modern training pipelines.

03

Regularization

Overfitting diagnosis · L1/L2 · Dropout · Max-Norm · Early Stopping

Diagnosing Overfitting — The Bias-Variance View

Underfitting (High Bias)

- Train acc low, Val acc low
- Model too simple or LR too small
- Both curves far from target

Fix: More capacity · Better optimizer · LR schedule

Good Fit

- Train acc ≈ Val acc (small gap)
- Val loss stabilizes or improves
- Gradient norms healthy

Fix: Maintain this setup · Monitor for drift

Overfitting (High Variance)

- Train acc high, Val acc diverges
- Val loss rises while train loss falls
- Large train/val gap

Fix: Dropout · L2 / AdamW · More data · Early Stop

Rule of thumb: check train/val gap first. Small gap + low accuracy → underfitting. Large gap → overfitting.

L1 and L2 Weight Regularization

L2 Regularization (Ridge / Weight Decay)

$$\mathcal{L} = \mathcal{L}_0 + \lambda \Sigma w_i^2$$

- Penalizes large weights → simpler model
- Gradient: $\partial\mathcal{L}/\partial w \mathrel{+}= 2\lambda w$ → shrinks weights proportionally
- Equivalent to weight decay under SGD; NOT under Adam → use AdamW
- Keras: Dense(..., kernel_regularizer=regularizers.l2(0.01))

Typical λ: 1e-4 to 1e-2

L1 Regularization (Lasso)

$$\mathcal{L} = \mathcal{L}_0 + \lambda \Sigma |w_i|$$

- Promotes sparse weights (exact zeros)
- Gradient: $\partial\mathcal{L}/\partial w \mathrel{+}= \lambda \cdot \mathrm{sign}(w)$ → constant push to zero
- Useful for feature selection; less common in deep MLPs
- Keras: Dense(..., kernel_regularizer=regularizers.l1(0.01))

Combine with L2 for Elastic Net: regularizers.l1_l2(l1=0.01, l2=0.01)

Dropout — Reducing Co-adaptation

How Dropout Works

- **At each training step:** randomly sets p% of activations to 0
- **Inverted dropout:** remaining activations scaled by 1/(1−p) to preserve expected value
- **At inference:** all neurons active; effectively an ensemble of 2^n sub-networks
- **Placement:** Dense → BN → Activation → Dropout (after activation, before next layer)
- **Typical rate:** rate=0.2–0.5 for hidden layers; lower for output-adjacent layers

Keras

```
from tensorflow.keras.layers import
    Dense, BatchNormalization,
    Activation, Dropout

model = Sequential([
  Dense(256, kernel_initializer=
    'he_normal'),
  BatchNormalization(),
  Activation('relu'),
  Dropout(rate=0.3),
  ...
])
```

Dropout rate is a hyperparameter — too high reduces capacity, too low has little effect. Tune with Keras Tuner (Section 04).

Max-Norm Constraint and Early Stopping

Max-Norm Constraint

Projects each neuron's weight vector so its L2 norm ≤ max_value after every update.

Works alongside any optimizer — independent of LR. Particularly useful with large LRs.

Keras: Dense(..., kernel_constraint=constraints.MaxNorm(max_value=2.0))

Typical values: max_value = 1–4 (start with 2.0).

Max-Norm does NOT replace L2/Dropout — it is complementary.

Early Stopping

Stop training when val_loss stops improving. Restore weights from the epoch with best val_loss.

Already used in our experiments! Prevents wasting compute and overfitting on noise.

Keras: EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

Note: do NOT use EarlyStopping with warmup schedules — fires during warmup phase.

Combine with LR schedule: schedule manages LR, ES handles stopping.

04

Hyperparameter Tuning

Keras Tuner · Random Search · Hyperband · Model selection

Keras Tuner — Automated Hyperparameter Search

Instead of manually trying values, Keras Tuner searches the hyperparameter space for you.

`RandomSearch`

Samples hyperparameter combinations uniformly at random.

Simple but effective for low-dimensional spaces. Good first step.

When: Few hyperparameters (< 5). Quick exploration.

`Hyperband`

Adaptive resource allocation: starts many trials with few epochs, keeps only promising ones.

Fast — often 3–10× more efficient than Random Search.

When: Large spaces, limited compute. Usually recommended default.

`BayesianOptimization`

Builds a probabilistic model of the objective function, suggests next point intelligently.

Slower to set up but very sample-efficient.

When: Expensive experiments. Many hyperparameters.

`pip install keras-tuner` · Runs in < 10 min on Colab T4 for small MLPs

Keras Tuner — Hyperband Example


```
import keras_tuner as kt

def build_model(hp):
    # Tunable hyperparameters
    units  = hp.Choice('units', [64,128,256])
    drop   = hp.Float('dropout', 0.0, 0.5, step=0.1)
    lr     = hp.Choice('lr', [1e-3, 5e-4, 1e-4])
    wd     = hp.Choice('wd', [1e-4, 1e-3, 0.0])

    model = Sequential([
        Flatten(), Dense(units,
kernel_initializer='he_normal'),
        BatchNormalization(), Activation('relu'),
        Dropout(drop), Dense(10, activation='softmax')
    ])
    model.compile(AdamW(lr, weight_decay=wd), ...
    return model
```

```
# Run Hyperband search
tuner = kt.Hyperband(
    build_model,
    objective='val_accuracy',
    max_epochs=30,
    directory='tuner_dir',
    project_name='cifar10'
)

tuner.search(
    x_tr, y_tr,
    validation_split=0.1,
    callbacks=[
        EarlyStopping('val_loss',
patience=5)
    ]
)

best =
tuner.get_best_hyperparameters()[0]
print(best.values)
```

05

CIFAR-10 Experiments

Full stack · Ablation study · Practical defaults · Decision guide

Full Stack — CIFAR-10: He + BN + AdamW + LR Schedule


```python
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense, Flatten, BatchNormalization, Activation
from tensorflow.keras.optimizers import AdamW
from tensorflow.keras.callbacks import LearningRateScheduler
```

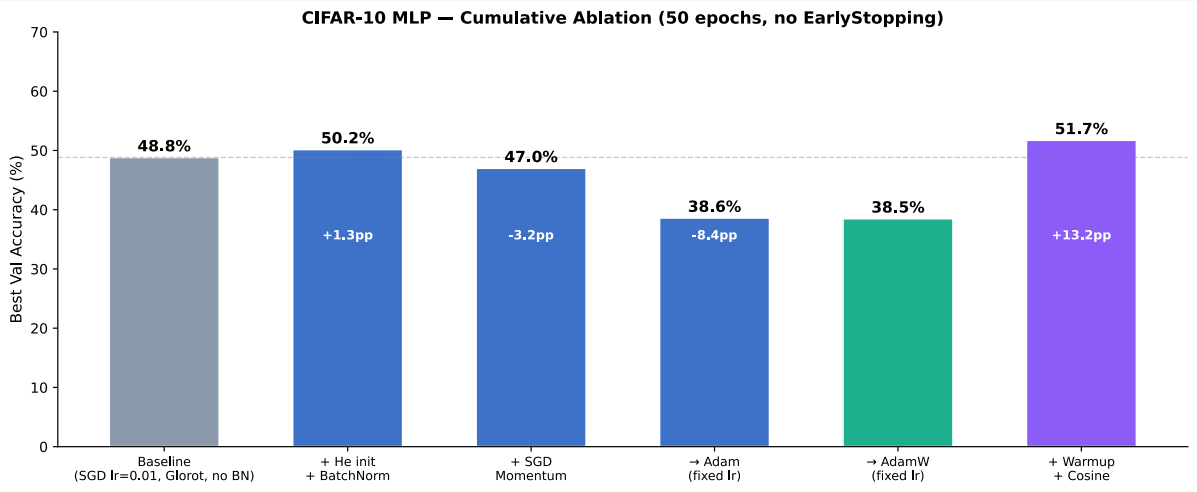
```python
model = Sequential([
    Input(shape=(32,32,3)),
    Flatten(),
    Dense(256, kernel_initializer='he_normal'),
    BatchNormalization(),
    Activation('relu'),
    Dense(128, kernel_initializer='he_normal'),
    BatchNormalization(),
    Activation('relu'),
    Dense(10, activation='softmax')
])
```

```python
opt = AdamW(
    learning_rate=1e-3,
    weight_decay=1e-4,
    clipnorm=1.0
)

model.compile(
    optimizer=opt,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    x_tr, y_tr,
    epochs=50, batch_size=256,
    validation_split=0.1,
    callbacks=[
LearningRateScheduler(lr_schedule)
    ]
)
```

Ablation Study — Optimizer & Scheduling Effects

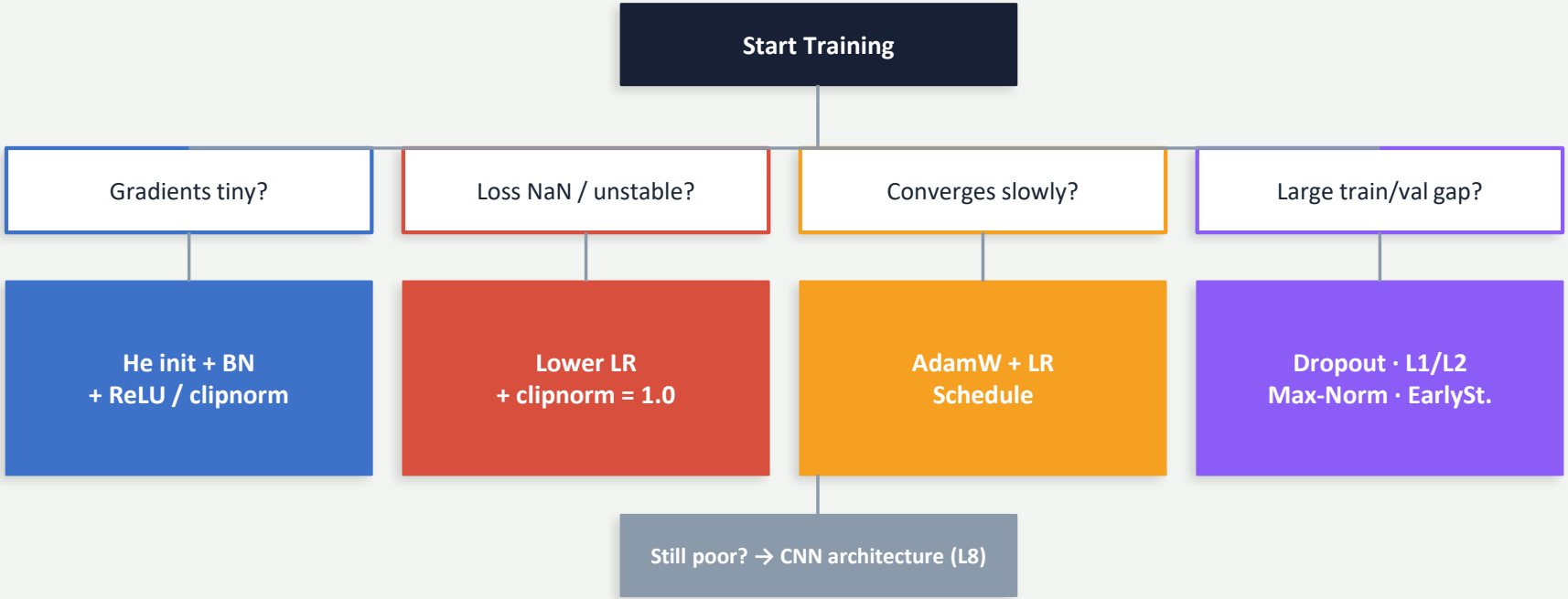


Illustrative MLP results on CIFAR-10. Values depend on seed, training length, and hyperparameters. This is an MLP baseline only — CNNs will far exceed these numbers.

Key Insights

- He Init + BN gives a small but real gain over the naive baseline
- Optimizer ranking is configuration-sensitive; in this fixed-LR comparison, Adam and AdamW perform similarly
- LR scheduling usually gives the clearest practical gain
- MLP baselines plateau in the low-50s here; CNNs will go much higher
- This is why optimization alone cannot replace architecture choice

Training Problem → Fix (Practical Guide)



Large train/val gap (overfitting) → also tune dropout rate with Keras Tuner (Section 04).

QUICK QUIZ

You train a CIFAR-10 MLP with He init + BatchNorm. Both train_acc and val_acc are low (~40%). Loss is decreasing steadily but very slowly. Gradients look healthy (mean_grad_norm ≈ 0.05). What is the best next step?

A Add Dropout(0.5) to each hidden layer — stronger regularization will fix the problem.

B Switch to AdamW or SGD+Momentum and apply a learning-rate schedule — slow but healthy loss decrease points to an optimization dynamics issue.

C Remove BatchNorm — it is slowing down training by stabilising the gradients too much.

D Add more layers immediately to increase model capacity.

Hint: both train_acc and val_acc are low — this is underfitting or optimization dynamics, not overfitting.

ANSWER

Answer: B — Switch to AdamW / SGD+Momentum + LR Schedule

Root cause:	Both train and val are low (~40%) — this is not overfitting. Dropout would make it worse. The model is learning, just too slowly.
Why B works:	Slow but healthy loss decrease signals a suboptimal optimization dynamics issue. Switching to AdamW or well-tuned SGD+Momentum, together with a learning-rate schedule (warmup + cosine), can significantly accelerate learning.
Why not A:	Dropout is a regularizer — it reduces model capacity. Adding it here would make underfitting worse, not better.
Why not C:	BatchNorm improves optimization stability and gradient flow. It does not cause slow convergence.
Why not D:	More layers will not help if the optimizer cannot drive learning effectively. Fix the dynamics first.

```
Pattern: both train & val low, gradients healthy  →  AdamW + LR schedule
```

APPENDIX

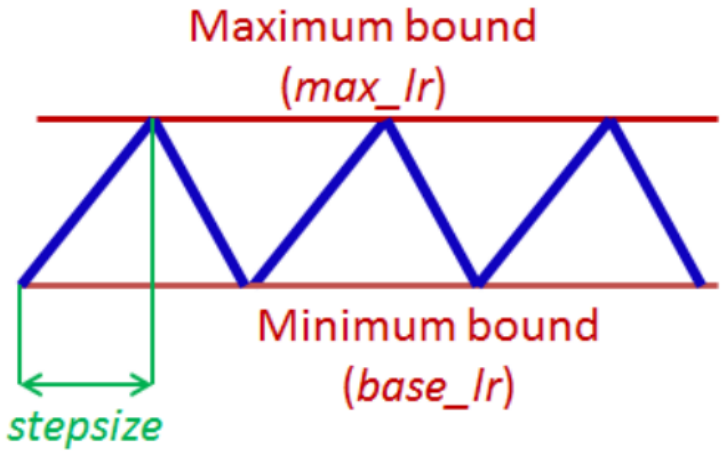
Cyclical Learning Rates · Colab Walkthrough

Cyclical Learning Rates (Smith, 2015)

Key Ideas

- Instead of monotonically decreasing the learning rate, let it cycle between a lower and an upper bound.
- The paper chooses a simple triangular policy because several cyclic shapes behaved similarly, and the triangle is the easiest to implement.
- A short LR range test—linearly increasing LR for a few epochs—gives practical lower and upper bounds for the cycle.
- CLR often matches or improves accuracy with far less tuning, and sometimes reaches the same accuracy in fewer iterations.

Choose a reasonable LR range once, then train by cycling within it.



*Triangular learning rate policy: the learning rate rises linearly to an upper bound, then falls linearly back to a lower bound. The **stepsize** parameter specifies the number of iterations in each half-cycle.*
arxiv:1506.01186

Run the Notebook — Colab Walkthrough

1 Section 1–2: Ravine intuition + CIFAR-10 setup

Run the ravine-loss example, then load and normalise CIFAR-10 for the experiments.

2 Section 3–4: Callbacks + cumulative ablation

Run the callback utilities and the cumulative ablation study. Compare He init + BN, Momentum, Adam, AdamW, and Warmup+Cosine.

3 Section 5–6: AdamW + LR schedules

Inspect AdamW vs Adam + L2 via the weight-norm plot, then compare fixed LR, ReduceLROnPlateau, Warmup+Cosine, and 1-cycle.

4 Section 7–8: Full stack + regularization

Run the full-stack CIFAR-10 baseline, then compare no regularization, Dropout, L2, and AdamW-style weight decay. Hyperparameter tuning is covered in Tutorial 2.